

# Midterm 1 Review Problems — Solutions

## Problem 1: Greedy Algorithms (stabbing intervals).

- (a) Take  $P = \{5, 10\}$ : the point 5 lies in  $[1, 5]$  and  $[4, 11]$ , and the point 10 lies in  $[8, 10]$  (and in  $[4, 11]$ ). A single point is impossible, because  $[1, 5]$  and  $[8, 10]$  are disjoint, so no point lies in both.
- (b) Sort the intervals by right endpoint,  $b_1 \leq \dots \leq b_n$ . Scan them in this order, keeping the last point added; whenever the current interval does not contain it, add the interval's right endpoint.
- ```

1: sort intervals so that  $b_1 \leq b_2 \leq \dots \leq b_n$ ;  $P \leftarrow \emptyset$ ;  $last \leftarrow -\infty$ 
2: for  $i = 1$  to  $n$  do
3:   if  $a_i > last$  then
4:     add  $b_i$  to  $P$ ;  $last \leftarrow b_i$ 
5:   end if
6: end for
7: return  $P$ 

```

The running time is  $O(n \log n)$ , dominated by the sort.

- (c) *Validity.* Consider interval  $I_i$  when it is scanned. If  $a_i > last$  we add  $b_i \in I_i$ . Otherwise  $a_i \leq last \leq b_i$  (the last point added is the right endpoint of an earlier interval, hence at most  $b_i$  by the sorting), so  $last \in I_i$ . Thus  $P$  stabs every interval.

*Optimality.* The points of  $P$  are increasing, as each new point is at least  $a_i > last$ . Let  $Opt$  be an optimal set. We prove by induction on  $k$  the statement

$$S(k) : \text{ for every } 1 \leq k \leq |P| : |Opt| \geq k \text{ and } Opt(k) \leq P(k).$$

*Base case* ( $k = 1$ ).  $P(1) = b_1$  is the smallest right endpoint.  $Opt$  must contain a point in  $I_1 = [a_1, b_1]$ , so  $|Opt| \geq 1$  and  $Opt(1) \leq b_1 = P(1)$ .

*Inductive step.* Assume  $S(k)$  and  $k + 1 \leq |P|$ . Let  $I = [a, b]$  be the interval that caused  $P(k + 1) = b$  to be added. Then  $a > P(k)$ . Since  $Opt(1) < \dots < Opt(k) \leq P(k) < a$ , none of  $Opt(1), \dots, Opt(k)$  lies in  $I$ . But  $Opt$  must stab  $I$ , so it has a point  $Opt(k') \in [a, b]$  with  $k' \geq k + 1$ . Hence  $|Opt| \geq k + 1$  and  $Opt(k + 1) \leq Opt(k') \leq b = P(k + 1)$ . This proves  $S(k + 1)$ .

*Conclusion.* If  $|P| > |Opt|$ , then  $S(|Opt| + 1)$  would give  $|Opt| \geq |Opt| + 1$ , a contradiction. So  $|P| \leq |Opt|$ , and  $P$  is optimal. ■

## Problem 2: Divide and Conquer ( $k$ -th smallest of two sorted arrays).

- (a) The sorted union is 10, 20, 30, 45, 46, 48, 62, 70, 80, 90, 100, 110, 120, so the 8-th smallest element is 70.
- Compare  $A[4] = 90$  with  $B[4] = 48$ . Since  $B[4] < A[4]$ , the elements smaller than  $B[4]$  are  $B[1], B[2], B[3]$  and at most  $A[1], A[2], A[3]$ , so  $B[4]$  has rank at most  $7 < 8$ , and so do  $B[1], B[2], B[3]$ . Hence  $B[1..4] = 20, 30, 45, 48$  can be discarded. The target now becomes the  $k = 8 - 4 = 4$ th smallest element of  $A \cup [70, 80, 100, 110]$ , which is again 70.

- (b) *Idea.* Compare the elements at position about  $k/2$  in each array. The array with the smaller such element cannot contain the  $k$ -th smallest element among its first  $k/2$  entries, so discard them and reduce  $k$  accordingly. Arrays are passed as offsets, so discarding costs  $O(1)$ .

```

1: procedure SELECT( $A, B, k$ )
2:   if  $A$  is empty then
3:     return  $B[k]$ 
4:   end if
5:   if  $B$  is empty then
6:     return  $A[k]$ 
7:   end if
8:   if  $k = 1$  then
9:     return  $\min(A[1], B[1])$ 
10:  end if
11:   $i \leftarrow \min(|A|, \lfloor k/2 \rfloor); \quad j \leftarrow \min(|B|, \lfloor k/2 \rfloor)$ 
12:  if  $A[i] < B[j]$  then
13:    return SELECT( $A[i+1..], B, k-i$ )
14:  else
15:    return SELECT( $A, B[j+1..], k-j$ )
16:  end if
17: end procedure

```

- (c) By induction on  $|A| + |B|$ , we show that SELECT returns the  $k$ -th smallest element of  $A \cup B$ , for  $1 \leq k \leq |A| + |B|$ .

*Base cases.* If  $A$  is empty the answer is  $B[k]$  (symmetrically for  $B$ ); if  $k = 1$  the answer is  $\min(A[1], B[1])$ .

*Inductive step.* Both arrays are nonempty and  $k \geq 2$ . By symmetry suppose  $A[i] < B[j]$ . The elements smaller than  $A[i]$  are  $A[1..i-1]$  and at most  $j-1$  elements of  $B$  (all of  $B[j..]$  exceed  $B[j] > A[i]$ ). So  $A[i]$  has rank at most  $(i-1) + (j-1) + 1 \leq 2\lfloor k/2 \rfloor - 1 \leq k-1$ . Every element of  $A[1..i]$  is at most  $A[i]$ , so none of them is the  $k$ -th smallest. We therefore delete  $i$  elements that all rank below the target, so its rank in the remaining set is exactly  $k-i$ . The call is valid:  $1 \leq k-i \leq (|A|-i) + |B|$ , and the total size has decreased, so the induction hypothesis gives the correct answer. ■

- (d) Each call costs  $O(1)$ . If  $i = \lfloor k/2 \rfloor$  (resp.  $j$ ), the new parameter is  $\lceil k/2 \rceil$ ; otherwise an array is exhausted and the next call is a base case. So

$$T(k) \leq T(\lceil k/2 \rceil) + c, \quad T(1) \leq c,$$

which gives at most  $\lceil \log_2 k \rceil + 2$  calls, i.e.  $T = O(\log k) \subseteq O(\log(m+n))$ . This is exponentially faster than the  $O(k)$  time of merging until  $k$  elements have been output.

### Problem 3: Dynamic Programming (coins 3, 7, 11).

- (a) Four coins summing to 24:  $11 + 7 + 3 + 3 = 24$  (also  $7 + 7 + 7 + 3 = 24$ ). The greedy rule fails: it takes 11 (remaining 13), then 11 again (remaining 2), and now no coin has value at most 2, so it gets stuck although 24 can be paid.
- (b) Let  $C[x]$  be the minimum number of coins summing to exactly  $x$ , with  $C[x] = \infty$  if impossible.

$$C[0] = 0, \quad C[x] = 1 + \min_{c \in \{3, 7, 11\}, c \leq x} C[x - c] \quad (x \geq 1),$$

where the minimum over an empty set (or over only  $\infty$  values) is  $\infty$ . Fill  $C[0], C[1], \dots, C[n]$  in increasing order; the answer is  $C[n]$  ( $\infty$  means impossible).

```

1:  $C[0] \leftarrow 0$ 
2: for  $x = 1$  to  $n$  do
3:    $C[x] \leftarrow 1 + \min\{C[x - c] : c \in \{3, 7, 11\}, c \leq x\}$   $\triangleright \infty$  if the set is empty
4: end for
5: return  $C[n]$ 

```

The running time is  $O(n)$  (three candidates per entry), using  $O(n)$  space.

- (c) Let  $OPT(x)$  be the true minimum number of coins summing to  $x$  ( $\infty$  if none). We show  $C[x] = OPT(x)$  by strong induction on  $x$ .

*Base case.*  $x = 0$ : the empty collection uses 0 coins, so  $OPT(0) = C[0] = 0$ .

*Inductive step* ( $x \geq 1$ ). Assume  $C[y] = OPT(y)$  for all  $y < x$ .

- ( $\geq$ ) Any collection of coins summing to  $x \geq 1$  is nonempty, so it contains some coin  $c \in \{3, 7, 11\}$  with  $c \leq x$ . Removing it leaves a collection summing to  $x - c$  with one fewer coin, so its size is at least  $1 + OPT(x - c) \geq 1 + \min_c OPT(x - c)$ . Hence  $OPT(x) \geq 1 + \min_c OPT(x - c)$ . In particular, if all  $OPT(x - c) = \infty$  (or no  $c \leq x$ ), no collection exists and  $OPT(x) = \infty$ .
- ( $\leq$ ) For each  $c \leq x$  with  $OPT(x - c) < \infty$ , adding one coin  $c$  to an optimal collection for  $x - c$  gives a collection for  $x$  of size  $1 + OPT(x - c)$ . Hence  $OPT(x) \leq 1 + \min_c OPT(x - c)$ .

So  $OPT(x) = 1 + \min_c OPT(x - c) = 1 + \min_c C[x - c] = C[x]$  by the induction hypothesis.

■